204. Count Primes

Medium | Topics | Companies | Hint

Given an integer `n`, return *the number of prime numbers that are strictly less than* `n`.

Example 1:

> **Input:** n = 10
> **Output:** 4
> **Explanation:** There are 4 prime numbers less than 10, they are 2, 3, 5, 7.

Example 2:

> **Input:** n = 0
> **Output:** 0

Example 3:

> **Input:** n = 1
> **Output:** 0

Constraints:

- `0 <= n <= 5 * 10^6`

DSA 11-11-26 / day-95

leetcode - 2204 - Count Primes

You are given an integer n, you need to return the total no. of prime numbers that are strictly smaller than n,

imp. obs
- 0 is not prime
- 1 " " "
- 2 " the smallest prime no.
- we count prime numbers less than n, not less than or equal to n.

approach 1 = brute force (checking every divisor)
for every number from 2 to n-1, check whether it is prime. To check whether a number is prime, try dividing it by every no. from 2 upto num-1. If any no. divides it completely, it is not prime. Otherwise it is prime.

```
class Solution { public:
    int countPrimes(int n) {
        int count = 0;
        for (int i = 2; i < n; i++) {
            bool isPrime = true;
            for (int j = 2; j < i; j++) {
                if (i % j == 0) {
                    isPrime = false;
                    break;
                }
            }
            if (isPrime) { count++; }
        }
        return count;
    }
};
```

we maintain a variable int count to store the total no. of prime numbers.

we check whether any no. from 2 to i-1 divides i, returns the remainder. if the remainder = 0, i divides i completely. so, i is not prime.

we traverse through every number starting from 2 till n-1. initially, we assume that the current no. is prime, if we find a divisor, we change it to false. mark the no. as non-prime & stop checking further divisors.

if no divisors were found, the no is prime, so increment the count, finally return the count.

Brute force (my own):

n = 10

number	divisors checked	prime?	count
0			
2	none	y	1
3	none	y	2
4	2	n	2
5	none	y	3
6	2	n	3
7	none	y	4
8	2	n	4
9	3	n	4

final answer = 4.

T.C. = $O(n^2)$ → we iterate through n numbers, and for each numbers, we may check almost n divisor.

S.C. = $O(1)$.
↳ we only use a few variable

not efficient bcuz we repeatedly perform unnecessary divisibility checks.

let's improve this!!!

Approach 2 - optimized prime checking (square root)

do we really need to check all numbers upto i-1 to determine whether i is prime?
↳ no, we only need to check divisors upto $\sqrt{i}$

why upto $\sqrt{i}$? for i=36, factor pairs of 36:-

1×36, 3×12, 6×6,
2×18, 4×9

notice, if we find a factor greater than $\sqrt{36}$, its corresponding factor is smaller than $\sqrt{36}$,
hence, for ex, 4 is smaller than $\sqrt{36}$
9 " great than $\sqrt{36}$,
therefore, if a no. has a factor greater than its square root, it must also have a corresponding factor smaller than its square root,
so, checking beyond $\sqrt{i}$ is unnecessary.

```
class Solution{ public:
    int countPrimes(int n){
        int count=0;
```

n = the upper limit given in the que.
i = the curr. no. whose primality we are checking.

```
        for(int i=2; i<n; i++){
            bool isPrime = true;
            for(int j=2; j*j<=i; j++){
                if(i%j==0){
                    isPrime = false;
                    break;
                }
            }
            if(isPrime){
                count++;}}
        return count;
    }};
```

only this changed for optimization.

inside loop first we check a no. then we decide whether to count it or not in next loop.

Previously:- $j < i$

__/__/____

Now:- $j * j <= i$

this is equivalent to checking divisors upto $\sqrt{i}$, without repeatedly calculating the square root,

Say here, $i = 25$,

we want to check whether 25 is prime or not, , isPrime = true;

now

j	j*j	25%j == 0	Action
2	4	1	continue
3	9	01	continue
4	16	1	continue
5	25	0	not prime.

At $j = 5$; $25 \% 5 == 0$

therefore; isPrime = false;

25 is not prime.

$t <= O(n\sqrt{n})$ → for every no., we check divisors upto its square root,

$s <= O(1)$

↳ we use constant extra space.

even

but can we do better?

yes, this is where the most imp. concept comes in.

Approach 3 - sieve of Eratosthenes instead of checking whether every no. is prime we could, alternate are numbers that are definitely not prime.

this technique is called sieve of Eratosthenes.

suppose: n = 10,

we have numbers: 0 1 2 3 4 5 6 7 8 9.

initially, we assume every number is prime, but we already know: • 0 is not prime
• 1 is not prime.

So, mark them as false,

Now; 0 1 2 3 4 5 6 7 8 9
F F T T T T T T T T

here, T means potentially prime,
F means not prime

now we start eliminating multiples,

S1 ⇒ consider 2, 2 is prime, its multiples are:
4, 6, 8,
all these numbers are divisible by 2, so they cannot be prime,
mark them false,

0 1 2 3 4 5 6 7 8 9
F F T T F T F T F F.

S2 ⇒ consider 3, 3 is still marked true, so it is prime,
it's multiples are, 6, 9, mark them false,
• 6 was already marked false in S1, so that's completely fine.

updated array →

0	1	2	3	4	5	6	7	8	9
F	F	T	T	F	T	F	T	F	F

S3 consider 4, 4 is already false, therefore, skip it

S4 → 5, 5 is greater than $\sqrt{10}$, there is no need to process it because all its smaller composite multiples have already been eliminated by smaller prime factors, the remaining true values are

2, 3, 5, →,

total, Answer = 4.

```
class Solution { public:
    int countPrimes(int n) {
        vector<bool> isPrime(n, true);
        if (n <= 2) return 0;
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; i++) {
            if (isPrime[i]) {
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
            }
        }
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (isPrime[i]) count++;
        }
        return count;
    }
};
```

We create a boolean array of size n. Initially, every element is true.

tells us whether the no. i is currently considered prime.

explicitly told, all are prime.

Such we only need to process prime numbers upto $\sqrt{n-1}$, any composite no. greater than $\sqrt{n}$ must have been eliminated by one of its smaller prime factors.

before eliminating multiples of i, we check whether i is still prime

mark all multiples of i as non-prime.

isPrime[j] = false; → we mark the current multiples as non-prime, bcz it has a divisor other than 1 & itself.

→ we initialize our answer.

int count = 0; → we iterate overall no. from 2 to n-1

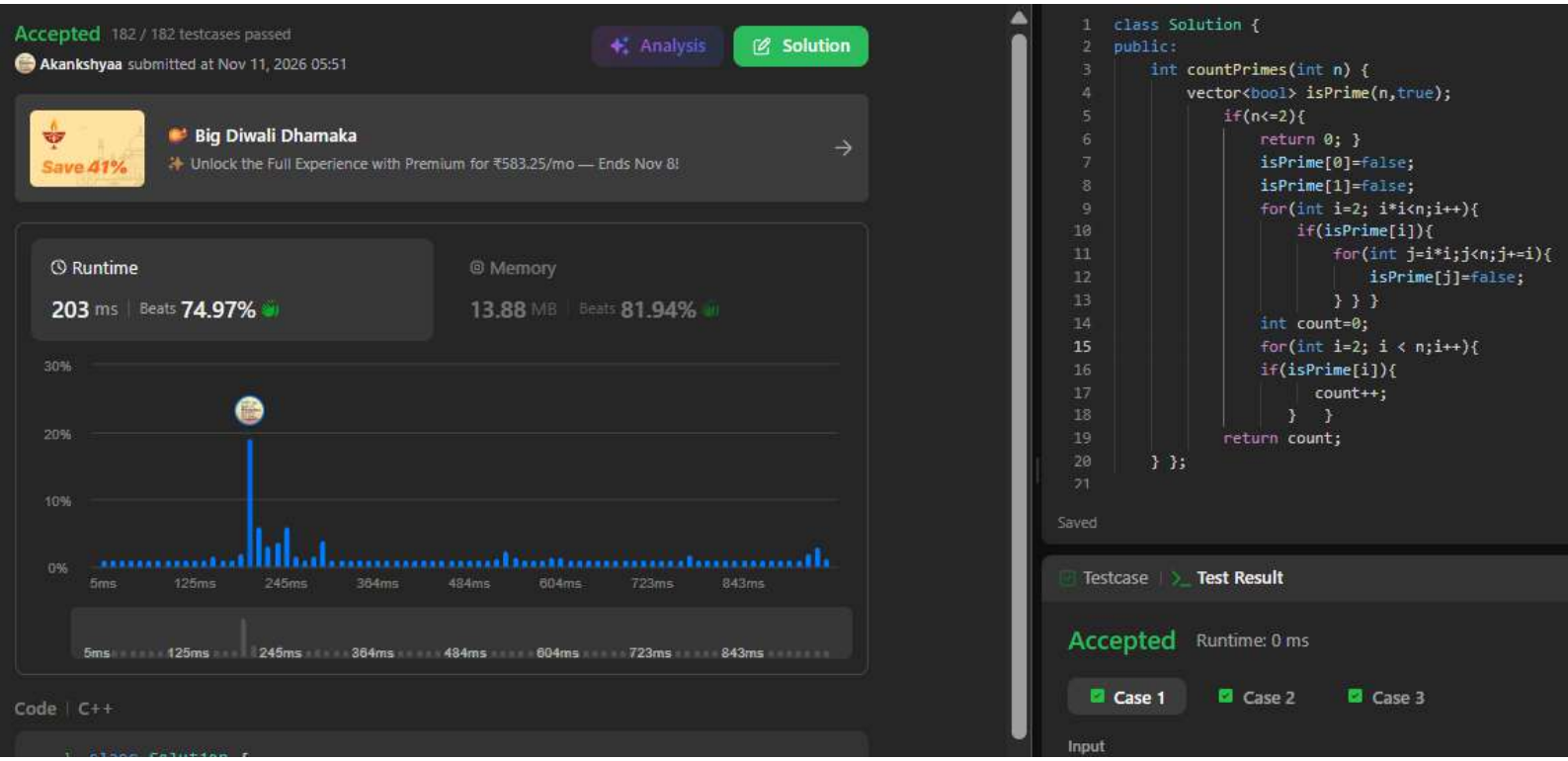
for (int i = 2; i < n; i++) {
    if (isPrime[i]) { if a number is still true, it is prime, increment the count; count, returns the total no. of prime no.
        count++;
    }
};

why int j = i * i, not 2 * i?

suppose, i = 5,
multiples of 5 are, 10, 15, 20, 25, 30, ...
notice, 10 is eliminated by 2,
15 " " " 3
20 " " " 4,
So, there is no need to process them again, the 1st multiple that needs to be considered is 5 * 5 = 25,
j += i means, keep adding i to j
for i = 5, j = 25, 30, 35, 40 ....

T.C = O(n log(log n)) → we eliminate multiples of prime no,
S.C = O(n)
for 2, n/2 multiples process.
for 3, n/3
n(1/2 + 1/3 + 1/5 ...)

Give create a boolean array of size to store whether each no. is prime


Accepted 182 / 182 testcases passed
Akankshyaa submitted at Nov 11, 2026 05:51
Analysis
Solution
Save 41%
Big Diwali Dhamaka
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!
Runtime
203 ms | Beats 74.97%
Memory
13.88 MB | Beats 81.94%
30%
20%
10%
0%
5ms
125ms
245ms
364ms
484ms
604ms
723ms
843ms
Code | C++
class Solution {
public:
int countPrimes(int n) {
vector<bool> isPrime(n,true);
if(n<=2){
return 0; }
isPrime[0]=false;
isPrime[1]=false;
for(int i=2; i*i<n;i++){
if(isPrime[i]){
for(int j=i*i;j<n;j+=i){
isPrime[j]=false;
} } }
int count=0;
for(int i=2; i < n;i++){
if(isPrime[i]){
count++;
} }
return count;
} };
Saved
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Case 3
Input